



Budakattu-Sante

Forest-to-Home Tribal Marketplace

Android App Development Project Report

Tribal Rural Development – Android Application

Project Title	Android App Development using GenAI – Budakattu-Sante (Rural Development) – I
Project Number	17
Platform	Android (Kotlin + XML Layouts)
Domain	Rural Development / Tribal Commerce / Social Empowerment
Category	Android Application with Room Database & WorkManager
Report Type	Internship / Academic Project Report

Prepared By

Arshid Ahmad Malik
USN: 1MJ23EC401
E-Mail: 1mj23ec401@mvjce.edu.in
MVJ College of Engineering
Department of Electronics and Communication Engineering
2025 – 2026

1. Introduction

Budakattu-Sante is an Android application developed to empower tribal artisans and forest-dwelling communities — known in Karnataka as Budakattu (forest people) — to sell their seasonal forest products and handcrafted goods directly to urban buyers, bypassing exploitative middlemen. The application name combines the Kannada word *Budakattu* (forest / tribal) with *Sante* (market / fair), reflecting its core purpose: to be the trusted digital marketplace for every tribal cooperative.

Developed using Kotlin with XML-based layouts, the app integrates Room Database for permanent offline-first inventory storage, WorkManager for background order and stock management tasks, and Glide/Coil for high-quality image loading of tribal craft listings. The application targets the informal tribal commerce ecosystem — a segment that has historically relied on weekly haat bazaars, roadside stalls, and government melas to reach buyers, with little transparency in pricing or supply-chain tracking.

The application addresses three critical needs of tribal cooperatives: allowing a cooperative leader to list seasonal forest products (wild honey, bamboo crafts, herbal oils, wild amla) with stock limits and Minimum Support Prices (MSP), enabling urban buyers to pre-order items before they are harvested, and maintaining a supply log that tracks which tribal family supplied which batch — ensuring fair, traceable, direct payment to the source producer.

Budakattu-Sante is designed to function in zero-network forest environments, with all inventory and order data stored offline using Room Database and synced when connectivity becomes available. The app targets the vast and underserved tribal community across Karnataka's forest districts — a community that produces unique, organic goods of significant value but has historically received only a fraction of the final market price due to deep middlemen supply chains.

2. Problem Statement

India's tribal artisan and forest-produce economy is one of the most exploited segments of the informal sector. Tribal communities produce unique, high-value goods — wild honey, bamboo crafts, herbal oils, medicinal roots — yet receive only a small fraction of the final retail price because of multi-layer middlemen who control access to urban buyers and pricing information. Several systemic problems affect this segment:

Challenge	Description
Middlemen Exploitation	Tribal artisans are forced to sell exclusively to local traders who buy at below-cost prices and resell to urban markets at 5-10x margins, denying producers fair income.
No Direct Market Access	Tribal communities in forest areas have no platform to reach urban buyers directly. Weekly haats and government melas offer limited reach and irregular opportunities.
No MSP Awareness	Artisans are unaware of government-approved Minimum Support Prices for forest produce (NTFP), making them vulnerable to systematic under-pricing by traders.
No Supply Traceability	There is no system to track which tribal family supplied a particular batch of honey or craft, making equitable and direct payment to individual producers impossible.
Pre-order Gap	Buyers cannot pre-order seasonal products before harvest, forcing artisans to produce speculatively, resulting in unsold stock, waste, and financial loss.
Zero-Network Constraints	Forest areas have no reliable internet connectivity, making cloud-first applications entirely unusable for inventory management at the point of production.

These interconnected challenges highlight the urgent need for a dedicated, offline-first tribal marketplace application that cooperative leaders can use with minimal technical knowledge, on low-cost Android smartphones, in zero-network forest environments.

3. Project Vision

Budakattu-Sante envisions a world where every tribal artisan — regardless of location, literacy level, or internet access — receives a fair price for their forest produce and handcrafted goods, with full traceability of supply and direct payment to the source family.

The project is built on four foundational pillars:

- **Accessibility** – The marketplace must function in zero-network forest environments. All core features — product listing, stock management, supply logging, and order recording — operate fully offline using Room Database, syncing when connectivity returns.
- **Simplicity** – The interface must be usable by tribal cooperative leaders who may have limited literacy. Large touch targets, audio-based descriptions for semi-literate users, earthy tones (green/brown), and visual-first product cards are non-negotiable.
- **Fairness** – Every transaction must be traceable to the tribal family who supplied the product. The Supply Log ensures that income flows directly and equitably to individual producers, eliminating the information asymmetry that enables exploitation.
- **Empowerment** – By displaying government-approved MSP (Minimum Support Price) data alongside product listings, the app arms artisans with price transparency, enabling them to negotiate fair deals and reject below-MSP offers.

Budakattu-Sante is not merely a product listing app; it is a digital mela — a fair marketplace that brings the forest to every home, removes exploitation from the supply chain, and gives tribal communities ownership of their own economic story.

4. Scope of the Application

4.1 Target Users

- Tribal cooperative leaders (Budakattu communities) across Karnataka's forest districts.
- Individual tribal artisans producing wild honey, bamboo crafts, herbal oils, and NTFP.
- Urban buyers seeking authentic, organic, directly-sourced tribal products.
- Government NTFP (Non-Timber Forest Produce) officers managing MSP compliance.
- NGOs and social enterprises supporting tribal livelihood programmes.

4.2 Platform

- Android smartphones with minimum API Level 21 (Android 5.0 Lollipop and above).
- Optimised for low-end and mid-range devices with limited RAM and storage.
- APK size kept minimal; Glide/Coil used for efficient image caching.

4.3 Offline Functionality

- All product listings and inventory data stored locally using Room Database — no server required for core operations.
- Supply Log entries, pre-order records, and stock updates work fully offline.
- Data sync to a remote server occurs automatically when network connectivity is detected.

4.4 Key Operational Scenarios

- A cooperative leader lists a new seasonal product (e.g., Wild Honey, 50 kg batch) with quantity, MSP, and source family details.
- An urban buyer browses the Forest Catalog, views product images and descriptions, and places a pre-order against available stock.
- The app enforces a Stock Limit — pre-orders beyond available quantity are blocked automatically.
- The Supply Log records which tribal family supplied each batch, the quantity, and the agreed price for payment traceability.
- MSP Info displays government-approved minimum prices alongside listings, enabling artisans to identify and refuse below-MSP offers.

5. Objectives

The project is aligned with the following measurable objectives:

- **Provide a fast and accurate product listing system** – Ensure that a new forest product can be listed within one screen interaction, with fields for product name, category, quantity (kg/units), price, MSP, image, and source family name.
- **Implement a pre-order system with Stock Limit logic** – Allow buyers to pre-order products before harvest. The system must enforce stock limits — preventing over-booking — and display remaining stock in real time after each order is placed.

- **Maintain a Supply Log with family-level traceability** – Record the tribal family name, quantity supplied, and price agreed for every batch entry, enabling direct and equitable payment to individual producers.
- **Display government-approved MSP data** – Integrate an offline MSP reference table for key NTFP products, displayed alongside each listing so artisans can immediately compare offers against government-approved minimum prices.
- **Ensure offline-first data persistence** – All product listings, orders, and supply log entries must survive application restarts, device reboots, and extended periods of zero-network inactivity using Room Database.
- **Support earthy, culturally appropriate UI** – Design the interface using forest green and earthy brown tones with visual-first product cards and audio description support for semi-literate tribal users.
- **Support data sync when connectivity returns** – Detect network availability via ConnectivityManager and automatically sync pending offline entries to a remote server when connection is restored.

6. Features

6.1 Core Features

Feature	Description
Forest Catalog	Browse seasonal forest products listed by the cooperative — Wild Honey, Bamboo Crafts, Herbal Oils, Wild Amla, and more. Each listing shows a high-quality product image (Glide/Coil), description, available quantity, asking price, and government MSP for comparison.
Pre-Order System	Urban buyers can pre-order items before they are harvested. The system validates each pre-order against remaining stock and blocks orders that exceed available quantity (Stock Limit logic). Confirmed pre-orders are stored in Room Database with buyer details, quantity, and order date.
Stock Limit Logic	Every product listing carries a maximum available quantity. As pre-orders are placed, the available stock is decremented in real time. When stock reaches zero, the product is marked 'Sold Out' and new pre-orders are rejected automatically.
Supply Log	Cooperative leaders log each supply batch with the tribal family name, product, quantity supplied, and agreed price. The log provides end-to-end traceability from forest floor to buyer doorstep, enabling direct payment to individual producer families.
MSP Info Screen	A dedicated screen displays government-approved Minimum Support Prices for key NTFP products. MSP data is stored offline and updated from a government feed when connectivity is available, giving artisans permanent access to fair-price reference data.
Offline-First Storage	Room Database (SQLite) stores all product listings, pre-orders, and supply log entries permanently on-device. Data is retained after app closure, device restart, or extended periods in zero-network forest environments.
Data Sync	ConnectivityManager detects network availability. WorkManager schedules a background sync task that uploads pending offline entries to the remote server when internet is detected, using ExistingWorkPolicy.KEEP to avoid duplicate uploads.
Cooperative Profile	Leaders save the cooperative's name, forest district, contact person, and UPI ID. The cooperative name appears in all product listings and is referenced in buyer order confirmations.
Settings Screen	Centralised settings for toggling data sync, managing cooperative profile, setting notification preferences for new pre-orders, and accessing MSP update controls.

6.2 Future Scope Features

- **UPI Payment Integration** – Allow buyers to complete payment directly within the app via UPI, with automatic payment confirmation to the source tribal family.
- **Multi-Language Support** – Extend the interface to Kannada, Hindi, Tamil, and local tribal dialects in addition to English.

- GenAI Product Description – Use a local LLM to auto-generate product descriptions in multiple languages from a photo and basic product details.
- Cloud Backup – Optional encrypted cloud sync via Firebase to protect cooperative inventory data against device loss.

- Buyer Rating System – Allow buyers to rate products and cooperatives, building trust and accountability in the marketplace.
- Biometric Authentication – Replace PIN login with fingerprint authentication on supported devices for cooperative leaders.

7. Functional Requirements

The following functional requirements define the core behaviours the system must support:

FR-1: Cooperative Profile Setup

- The system shall display a profile setup screen on first launch, prompting the cooperative leader to enter the cooperative name, forest district, contact person, and UPI ID.
- Profile data shall be stored in SharedPreferences and persist across all sessions.
- The cooperative name shall appear in the main dashboard header and in all product listings.
- The leader shall be able to update the cooperative profile at any time via the Settings screen.

FR-2: Product Listing

- The system shall provide input fields for product name, category, available quantity (kg/units), asking price (Rs.), government MSP (Rs.), source family name, and product image.
- The system shall validate that product name, quantity, and price are not empty and that quantity/price are positive numeric values before saving.
- On successful save, the system shall store the listing in Room Database with a timestamp and refresh the Forest Catalog immediately.
- The cooperative leader shall be able to edit or delete any product listing via a long-press action.

FR-3: Pre-Order System

- The system shall allow buyers to place a pre-order by entering their name, phone number, and desired quantity against a listed product.
- The system shall validate that the requested quantity does not exceed the remaining available stock before confirming the order.
- On successful pre-order, the system shall decrement the product's available stock in Room Database and reflect the update in the Forest Catalog in real time.
- When available stock reaches zero, the system shall mark the product as 'Sold Out' and reject further pre-orders.

FR-4: Supply Log

- The system shall provide an input form for logging a supply batch with fields for tribal family name, product name, quantity supplied (kg/units), agreed price (Rs.), and date.
- All supply log entries shall be stored in Room Database with an auto-generated timestamp.
- The leader shall be able to view the full supply log, filtered by product or family name.
- The system shall display the total quantity and total value supplied per family on request.

FR-5: MSP Info

- The system shall maintain an offline MSP reference table for key NTFP products including Wild Honey, Bamboo Products, Herbal Oils, Amla, and Tamarind.
- MSP data shall be displayed alongside each product listing for direct comparison with the asking price.
- The system shall flag any listing where the asking price is below the government MSP with a visual warning indicator.
- MSP data shall be updatable from a government feed URL when network connectivity is available.

FR-6: Offline Data Sync

- The system shall use ConnectivityManager to detect network availability.
- WorkManager shall schedule a background task to upload all pending offline entries (listings, orders, supply log) to the remote server when connectivity is detected.
- The sync task shall use ExistingWorkPolicy.KEEP to prevent duplicate uploads.
- The system shall display a sync status indicator in the dashboard header showing last sync time.

FR-7: Notifications

- The system shall notify the cooperative leader via a push notification when a new pre-order is placed against any of their product listings.
- The leader shall be able to toggle pre-order notifications on or off independently from the Settings screen.
- Notifications shall include the buyer name, product name, quantity ordered, and order date.

8. Non-Functional Requirements

Category	Requirement	Acceptance Criterion
Performance	Load Time	Forest Catalog must fully render within 800 ms of launch on a mid-range Android device.
Performance	Listing Save	A new product listing must be saved and reflected in the catalog within 500 ms of tapping Save.
Usability	UI Simplicity	A cooperative leader must be able to list a new product within 3 taps from the home screen.
Usability	Legibility	All primary text must use a minimum font size of 13sp. Product price values must use a minimum of 20sp.
Reliability	Data Persistence	100% of product listings and supply log entries must be retained after app closure, device restart, and 30+ days of offline use.
Reliability	Crash Rate	Application crash rate must not exceed 1% during standard listing, ordering, and supply logging flows.
Accessibility	Touch Targets	All interactive elements must have a minimum touch target size of 48x48dp per Material Design guidelines.
Compatibility	Android Versions	Application must support Android 5.0 (API 21) through Android 14 (API 34).
Security	Data Privacy	All cooperative and supply chain data remains on-device unless explicitly synced. No data is transmitted without user consent.
Offline Support	Core Features	All core features (listing, pre-ordering, supply logging, MSP display) must function without any network connection.

9. System Architecture

Budakattu-Sante follows an offline-first, client-primary architecture. All core data processing, storage, and business logic occur on-device. A lightweight remote sync layer operates in the background via WorkManager when network connectivity is available, but all features remain fully functional without it. **9.1 Technology Stack**

Layer	Technology	Purpose
Language	Kotlin	Primary development language; concise, null-safe, fully interoperable with Android SDK.

UI Framework	XML Layouts + AppCompat	Traditional View-based UI with RecyclerView, CardView, and Material components for broad device compatibility.
Image Loading	Glide / Coil	Efficient, cache-aware image loading for tribal product photos. Supports placeholder and error images for offline use.
Local Database	Room Database (SQLite)	ACID-compliant ORM over SQLite. Stores product listings, pre-orders, and supply log entries permanently on-device.
Background Tasks	WorkManager	Schedules and executes offline data sync tasks reliably, surviving device reboots and app closures.
Network Detection	ConnectivityManager	Monitors network state changes to trigger data sync automatically when connectivity is restored.
Notifications	NotificationManager + AlarmManager	Delivers pre-order alerts to cooperative leaders. Supports Android 8+ notification channels.
Session Storage	SharedPreferences	Stores cooperative profile data, sync preferences, and app settings as lightweight key-value pairs.
Build System	Gradle (Groovy DSL)	Dependency management, build configuration, and APK generation including kotlin-kapt for Room annotation processing.
Min. SDK	API Level 21	Targets approximately 98% of active Android devices globally, including low-cost smartphones prevalent in forest and tribal areas.

9.2 Architectural Overview

The application follows a simplified MVC (Model-View-Controller) pattern:

- **Model Layer** – The data package contains Room @Entity classes (Product, PreOrder, SupplyEntry, MspReference), @Dao interfaces (ProductDao, PreOrderDao, SupplyDao, MspDao), and the @Database singleton (AppDatabase). This layer owns all data access logic and migration handling.
- **View Layer** – XML layout files define the UI (activity_main.xml, activity_catalog.xml, activity_preorder.xml, activity_supply_log.xml, activity_msp.xml, activity_settings.xml, list_item_product.xml, list_item_supply.xml). Earthy green/brown drawables provide culturally appropriate visual styling.
- **Controller Layer** – Activity classes (MainActivity, CatalogActivity, PreOrderActivity, SupplyLogActivity, MspActivity, SettingsActivity) orchestrate user interaction, invoke database operations on background threads via Executors, and update the UI on the main thread via runOnUiThread.

• **Background Workers** – SyncWorker (Worker) handles offline-to-online data synchronisation. OrderNotificationReceiver (BroadcastReceiver) delivers pre-order alerts to the cooperative leader.

9.3 Database Design

The Room database contains four primary tables:

Table: products

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY, AUTO	Auto-generated unique identifier for each product listing.
productName	TEXT	NOT NULL	Name of the forest product (e.g., Wild Honey, Bamboo Craft).
category	TEXT	NOT NULL	Product category (Forest Produce / Handicraft / Herbal).
quantityKg	REAL	NOT NULL	Available quantity in kg or units (stored as Double).
priceRs	REAL	NOT NULL	Cooperative asking price in Indian Rupees.
mSPRs	REAL	DEFAULT 0	Government Minimum Support Price for reference.
sourceFamilyName	TEXT	NOT NULL	Name of the tribal family who supplied this batch.
imageUri	TEXT	DEFAULT "	URI of the product image stored on-device.
timestamp	TEXT	NOT NULL	Formatted date and time of listing creation.

Table: pre_orders

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY, AUTO	Auto-generated unique identifier.
productId	INTEGER	FOREIGN KEY	References products.id for order-product linkage.
buyerName	TEXT	NOT NULL	Name of the urban buyer placing the pre-order.
buyerPhone	TEXT	DEFAULT "	Buyer's phone number for order confirmation.
quantityOrdered	REAL	NOT NULL	Quantity pre-ordered (kg/units).
orderDate	TEXT	NOT NULL	Formatted date of pre-order placement.
synced	INTEGER	DEFAULT 0	0 = pending sync; 1 = uploaded to remote server.



Table: supply_log

Column	Type	Constraint	Description
productName	TEXT	NOT NULL	Product name supplied in this batch.
quantitySupplied	REAL	NOT NULL	Quantity supplied in kg or units.
agreedPriceRs	REAL	NOT NULL	Price agreed with the family for this batch.
supplyDate	TEXT	NOT NULL	Formatted date of supply entry.

10. User Flow

The application is designed for two primary user roles: the Cooperative Leader (supply side) and the Urban Buyer (demand side). The complete user journey is described below:

#	Screen	User Action & System Response
1	Profile Setup	App launches. If first use, cooperative leader enters cooperative name, district, contact, and UPI ID. On subsequent launches, the home dashboard is shown directly.
2	Home Dashboard	Leader sees the Forest Catalog with all active product listings, stock status cards, and a quick-access FAB to add a new listing.
3	Add Product Listing	Leader fills in product name, category, quantity, price, MSP, source family name, and uploads a photo. Taps Save. Listing is stored in Room DB and appears in the catalog immediately.
4	Browse Catalog (Buyer)	Buyer browses the Forest Catalog. Each card shows product image, name, available stock, asking price, and government MSP. Buyer taps a product to view full details.
5	Place Pre-Order	Buyer enters name, phone number, and desired quantity. The system validates against available stock. On confirmation, stock is decremented and the pre-order is saved to Room DB.
6	Stock Limit Enforcement	If a buyer attempts to order more than available stock, the system displays an error: 'Only X kg available.' The order is blocked until quantity is reduced.
7	Supply Log Entry	Leader logs a new supply batch: family name, product, quantity, price, date. Entry is saved to Room DB and appears in the Supply Log with a timestamp.
8	MSP Info	Leader navigates to MSP screen. Government-approved minimum prices for all listed NTFP products are displayed. Any listing below MSP is flagged with a warning icon.
9	Data Sync	When connectivity is detected, WorkManager uploads all pending offline records (listings, orders, supply log) to the remote server. Sync status updates in the dashboard header.
10	Settings	Leader taps the menu → Settings. Manages cooperative profile, toggles pre-order notifications, configures sync preferences, and accesses MSP update controls.

11. Technical Implementation

11.1 Room Database & Migration

Product listings, pre-orders, and supply log entries are stored using the Room persistence library, which provides a type-safe abstraction over SQLite. Each DAO interface exposes standard CRUD operations plus domain-specific queries: `ProductDao.getAvailableProducts()` filters out sold-out items; `SupplyDao.getByFamily()` groups supply entries by family name for payment calculation. A database migration (version 1 → version 2) adds the synced column to `pre_orders` using an `ALTER TABLE` statement, ensuring that users who upgrade do not lose their order history.

11.2 Stock Limit Logic

When a buyer submits a pre-order, `PreOrderActivity` queries the current available quantity for the selected product using `ProductDao.getQuantityById()`. If the requested quantity exceeds the available stock, a validation error is displayed and the order is rejected before any database write occurs. On successful validation, the pre-order is inserted and the product's `quantityKg` is decremented atomically within a Room transaction block (`@Transaction` annotation on `AppDatabase`), preventing race conditions in multi-user scenarios.

11.3 WorkManager for Offline Sync

`SyncWorker` extends `Worker` and implements `doWork()` to upload all Room Database records where `synced == 0` to the remote server via an HTTP POST request. It is enqueued as a `OneTimeWorkRequest` triggered by a `ConnectivityManager` network callback, using `WorkManager.enqueueUniqueWork()` with `ExistingWorkPolicy.KEEP` to prevent duplicate sync jobs. On successful upload, the worker marks each record as `synced == 1` in Room Database.

11.4 Glide / Coil for Image Loading

Product listing images are captured from the device camera or gallery and stored as local file URIs in Room Database. Glide is used to load images into product cards with a placeholder drawable (earthy brown pattern) shown during loading and an error drawable when the image file is unavailable. Glide's disk cache ensures that product images remain visible in the catalog even after the source file is moved or the app is restarted.

11.5 Background Threading

Room does not permit database operations on the main (UI) thread. All database calls are executed using `Executors.newSingleThreadExecutor()`. Results are posted back to the UI thread via `runOnUiThread()`, which updates the `RecyclerView` adapter and stock summary cards. This pattern avoids the need for `LiveData` or coroutines while maintaining thread safety across all Activity classes.

11.6 MSP Reference Table

Minimum Support Price data for key NTFP products is pre-loaded into the `msp_reference` table via Room's `RoomDatabase.Callback.onCreate()` method, which populates the table with a default dataset on first install. When network is available, `MspUpdateWorker` fetches an updated MSP JSON feed from a government endpoint and refreshes the local table, ensuring artisans always have access to the most current price reference.

11.7 Cooperative Profile & Notifications

The cooperative's name, district, and UPI ID are saved in SharedPreferences under CoopProfilePrefs. The cooperative name is displayed in the main dashboard header and in all product listings. When a new pre-order is placed, OrderNotificationReceiver builds and posts a NotificationCompat notification containing the buyer name, product name, and quantity ordered. On Android 8.0 and above, a dedicated notification channel (channel_orders) is created before posting.

12. Project File Structure

File / Directory	Purpose
app/build.gradle	App-level Gradle config: plugins, SDK versions, Room, WorkManager, Glide/Coil, and kapt dependencies.
data/Product.kt	Room @Entity data class representing a single forest product listing.
data/PreOrder.kt	Room @Entity data class representing a buyer pre-order with sync status flag.
data/SupplyEntry.kt	Room @Entity data class representing a tribal family supply batch log entry.
data/MspReference.kt	Room @Entity data class storing government MSP values for NTFP products.
data/ProductDao.kt	Room @Dao interface: getAll, getAvailableProducts, getQuantityById, insert, update, delete.
data/PreOrderDao.kt	Room @Dao interface: getAll, getUnsyncedOrders, insert, markSynced, delete.
data/SupplyDao.kt	Room @Dao interface: getAll, getByFamily, insert, delete operations for supply log.
data/MspDao.kt	Room @Dao interface: getAll, getByProduct, insert, updateAll for MSP reference data.
data/AppDatabase.kt	Room @Database singleton with version management, migration definitions, and onCreate pre-population of MSP data.
MainActivity.kt	Home dashboard: Forest Catalog RecyclerView, stock summary cards, FAB to add listing, navigation menu.
CatalogActivity.kt	Full product catalog view with search, filter by category, and product detail navigation.
PreOrderActivity.kt	Pre-order form: buyer details, quantity input, stock limit validation, order confirmation.
SupplyLogActivity.kt	Supply log screen: batch entry form, family-wise supply history, total value calculator.

MspActivity.kt	MSP reference screen: NTFP product list with government minimum prices and below-MSP warnings.
SettingsActivity.kt	Settings for sync preferences, cooperative profile management, and notification toggles.
SyncWorker.kt	WorkManager Worker: uploads pending offline records to remote server, marks synced on success.
OrderNotificationReceiver.kt	BroadcastReceiver: builds and posts pre-order notifications to the cooperative leader.

File / Directory	Purpose
ProductAdapter.kt	Custom RecyclerView.Adapter inflating list_item_product.xml with Glide image loading.
SupplyAdapter.kt	Custom RecyclerView.Adapter inflating list_item_supply.xml for supply log entries.
res/layout/activity_main.xml	Dashboard layout: cooperative header, stock summary cards, Forest Catalog RecyclerView, and FAB.
res/layout/list_item_product.xml	Product card: Glide image, product name, stock badge, price, MSP indicator.
res/layout/activity_supply_log.xml	Supply log layout: entry form and scrollable family-wise log list.
res/layout/activity_msp.xml	MSP screen layout: NTFP product list with price columns and warning icons.
res/drawable/*.xml	Shape drawables: earthy green/brown card backgrounds, forest-themed gradient headers, badge shapes.
res/menu/main_menu.xml	Overflow menu items: Supply Log, MSP Info, Settings.
AndroidManifest.xml	App manifest: permissions, activity declarations, receiver registration, launcher activity.

13. Permissions & Privacy

Permission	Required For	Notes
INTERNET	Remote data sync	Required for uploading offline entries to the server and fetching MSP updates. Core features work without it.
ACCESS_NETWORK_STATE	Connectivity detection	Used by ConnectivityManager to detect network availability and trigger sync worker.
POST_NOTIFICATIONS	Pre-order alert notifications	Required on Android 13+. Cooperative leader prompted on first order notification.
SCHEDULE_EXACT_ALARM	Scheduled sync tasks	Required on Android 12+. User may need to grant via system settings.
READ_EXTERNAL_STORAGE / READ_MEDIA_IMAGES	Product image selection	Required for accessing gallery images for product listings (API-level conditional handling).
CAMERA	Product photo capture	Required for taking product photos directly within the listing form.
RECEIVE_BOOT_COMPLETED	Restart sync worker after reboot	Ensures pending sync tasks are re-queued after a device restart.

No personal user data is transmitted to any external server without explicit user consent and active network connectivity. All product listings, supply log entries, cooperative profile information, and pre-order records remain on-device during offline operation. The application does not collect analytics or usage statistics. Buyer phone numbers are used solely for the purpose of order confirmation communications at the cooperative leader's request.

14. Impact Goals

14.1 Tribal Empowerment

By enabling tribal artisans to list their products directly and reach urban buyers without middlemen, Budakattu-Sante directly increases the income of forest-dwelling communities. Artisans who previously received 10-20% of the final retail price can now negotiate and transact at market value, improving working capital, reducing debt dependency, and strengthening economic self-sufficiency.

14.2 Vocal for Local

The Forest Catalog brings unique, organic, sustainably sourced tribal products — wild honey, bamboo crafts, herbal oils — to the attention of urban buyers who actively seek authentic alternatives to mass-produced goods. Budakattu-Sante is a digital expression of the Vocal for Local movement, connecting the forest economy to the global market while preserving indigenous production practices.

14.3 Economic Inclusion

By operating offline-first and targeting API 21 devices, the app brings tribal cooperatives in zero-network forest areas into the digital economy for the first time. Financial inclusion does not require broadband internet; it requires an app that works on a basic smartphone in the middle of a forest.

14.4 Supply Chain Transparency

The Supply Log provides end-to-end traceability from forest floor to buyer doorstep. Every batch is linked to the tribal family who produced it, enabling direct and equitable payment distribution. This transparency reduces disputes, builds buyer trust in the product's authenticity, and creates an auditable record for NGO and government oversight.

14.5 MSP Awareness & Fair Pricing

By displaying government-approved Minimum Support Prices alongside every listing, the app arms artisans with the price transparency they need to identify and refuse exploitative offers. Systematic access to MSP data shifts the power dynamic in price negotiations from traders to producers — a small data change with significant economic consequences.

14.6 Scalability of Impact

The modular architecture of Budakattu-Sante — with separate data, activity, and worker layers — supports rapid expansion. Future versions can add UPI payment integration, multi-language support, GenAI product descriptions, and buyer rating systems without requiring a fundamental redesign of the existing codebase.

15. Success Criteria

The project will be considered successfully completed when the following criteria are met:

#	Success Criterion	Measurement	Priority
1	App loads and reaches the Forest Catalog within 800 ms on a reference device (Redmi 9A or equivalent).	Android Profiler measurement	Critical
2	A new product listing can be created within 3 taps from the home screen.	Manual UI walkthrough	Critical
3	Stock Limit logic correctly blocks pre-orders that exceed available quantity.	Manual stock boundary test	Critical
4	All product listings and supply log entries are retained after device reboot and 30+ days of offline use.	Room DB inspection test	Critical
5	The app must allow offline data entry that syncs correctly when the network becomes available.	Airplane Mode + connectivity test	Critical
6	Supply Log correctly records family name, quantity, and price for every batch entry.	Manual log validation	High
7	MSP Info displays correct government prices and flags below-MSP listings with a warning indicator.	Manual MSP verification	High
8	App does not crash during a complete walkthrough of all screens and features.	Manual regression testing	High
9	Pre-order notification is received by the cooperative leader within 5 seconds of order placement.	Manual notification test	High
10	UI uses earthy tones (Green/Brown) consistently across all screens to reflect tribal culture.	Design review against style guide	Medium

Conclusion

Budakattu-Sante represents a focused and socially meaningful application of Android development in the domain of tribal rural development and forest-produce commerce. By addressing a clear and well-defined problem — the lack of a reliable, offline-first, and fair digital marketplace for India's tribal artisans — the project delivers measurable practical value alongside significant technical merit.

The application's architecture prioritises offline-first reliability, supply-chain transparency, and cultural accessibility above all else. Every design decision — from Room Database for zero-network inventory management, to WorkManager for resilient data synchronisation, to Stock Limit logic for pre-order fairness, to MSP Info for price transparency — is motivated by a single question: will this work for a tribal cooperative leader standing in a forest clearing with a basic smartphone and no internet connection?

The project also establishes a strong technical foundation for future expansion. The separation of data, view, and controller layers supports the addition of new features — UPI payment integration, GenAI product descriptions, multi-language support, buyer rating systems — without requiring a fundamental redesign of the existing codebase. The Room migration framework ensures that future schema changes can be deployed to existing users without data loss.

In conclusion, Budakattu-Sante is not simply a technical deliverable — it is a small but meaningful step toward economic justice, digital inclusion, and fair trade for the millions of tribal artisans who are the original custodians of India's forests and the makers of its most authentic goods.